

Documentation du Projet SecuScanPro

1. Présentation du Projet

1.1 Objectif

SecuScanPro est un outil d'analyse de sécurité web professionnel développé en Go. Il est conçu pour identifier automatiquement les vulnérabilités potentielles dans les applications web, en mettant l'accent sur la détection des injections SQL et la vérification des protections CSRF. L'application dispose d'une interface graphique moderne et intuitive facilitant son utilisation par des professionnels de la sécurité.

1.2 Fonctionnalités Principales

SecuScanPro intègre plusieurs fonctionnalités essentielles pour l'analyse de sécurité :

- Un crawler web intelligent qui analyse automatiquement la structure des pages web
- Un système de détection des formulaires et des points d'entrée sensibles
- Des tests d'injection SQL pour les principales bases de données (MySQL, PostgreSQL, MSSQL, Oracle)
- Une vérification automatique des protections CSRF dans les formulaires
- Une interface graphique moderne développée avec Fyne
- Un système de rapports détaillés avec stockage persistant
- Un suivi en temps réel de la progression des analyses
- Une visualisation statistique des résultats de sécurité

1.3 Structure du Projet

Le projet suit une architecture modulaire bien organisée :

```
SecuScanPro/
├── cmd/           # Point d'entrée de l'application
├── internal/      # Code interne de l'application
│   ├── config/   # Configuration et constantes
│   ├── crawler/  # Module d'analyse des pages web
│   ├── model/    # Définitions des structures de données
│   ├── security/ # Implémentation des tests de sécurité
│   ├── storage/  # Gestion des rapports
│   └── ui/       # Interface utilisateur
├── reports/      # Stockage des rapports générés
└── .gitignore    # Configuration Git
```

2. Organisation du Travail

2.1 Architecture et Design

L'architecture du projet est basée sur plusieurs principes clés :

```

```go
// Exemple de la structure principale d'un rapport
type SecurityReport struct {
 ID string `json:"id"`
 URL string `json:"url"`
 Date string `json:"date"`
 Results []PageElement `json:"results"`
}
```

```

2.2 Interface Utilisateur

L'interface graphique a été développée avec Fyne, offrant une expérience utilisateur moderne et responsive. Elle comprend :

- Une fenêtre principale avec des options d'analyse
- Un système de progression en temps réel des analyses
- Une visualisation détaillée des rapports avec statistiques
- Une interface de test d'injection SQL interactive
- Un questionnaire de rapports avec historique

2.3 Gestion des Données

Le système de stockage utilise un format JSON structuré pour les rapports :

- Stockage persistant dans le dossier `reports/`
- Nommage unique des rapports basé sur des timestamps
- Structure de données optimisée pour les analyses de sécurité

3. Difficultés Rencontrées et Solutions

3.1 Défis Techniques

1. Gestion de la Concurrency

Solution : Utilisation efficace des goroutines et des channels pour :

- Le suivi en temps réel de la progression
- L'exécution parallèle des tests de sécurité
- La communication entre l'interface et le moteur d'analyse

2. Précision des Tests

Solution : Implémentation d'une approche multi-niveaux :

- Détection intelligente des points d'injection
- Validation des résultats pour minimiser les faux positifs
- Tests adaptés par type de base de données

3. Performance du Crawler

Solution : Optimisation des analyses avec :

- Parsing HTML efficace avec ``golang.org/x/net/html``
- Gestion mémoire optimisée des rapports
- Traitement asynchrone des requêtes

4. Analyse de l'Utilisation de Go

4.1 Avantages de Go pour le Projet

1. Performance et Concurrency

- Goroutines pour les analyses parallèles
- Channels pour la communication inter-processus
- Performance native excellente pour le crawling

2. Bibliothèques Standards Robustes

- ``net/http`` pour les requêtes web
- ``encoding/json`` pour la gestion des rapports
- ``html`` pour le parsing des pages

3. Interface Graphique avec Fyne

- Création d'interfaces modernes
- Gestion native des événements
- Compatibilité cross-platform

4.2 Apprentissages sur Go

L'utilisation de Go a permis de développer des compétences importantes :

- Gestion efficace de la concurrence
- Structuration modulaire du code
- Traitement robuste des erreurs
- Développement d'interfaces graphiques natives

5. Conclusion

5.1 Réalisations

Le projet SecuScanPro démontre la capacité de Go à créer des outils de sécurité professionnels avec :

- Une architecture robuste et maintenable
- Des performances excellentes
- Une interface utilisateur moderne
- Une gestion efficace des analyses de sécurité

5.2 Perspectives d'Évolution

Le projet peut être enrichi avec :

- Support de nouveaux types de tests de sécurité
- Amélioration des algorithmes de détection

- Intégration d'analyses de vulnérabilités supplémentaires
- Extension des capacités de reporting